ASI

July 1, 2025

1 A Roadmap to Superintelligence: Modular Testing

1.1 Module 1: Recursive Cellular-Attention-Logic Hybrid

1.1.1 Phase 1: Base CA Engine

[7]:
```
# Cell 1: Imports and device selection
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f" Using device: {device}")
```

```
Using device: cuda
```

[8]:
```
# Cell 2: Core CA class with rule mutation, input injection, and metrics

class CellularAutomaton2D:
    def __init__(self, grid_size=(64, 64), batch_size=1, kernel=None,␣
 ↪threshold=3):
        self.grid_size = grid_size
        self.batch_size = batch_size
        self.threshold = threshold
        self.kernel = kernel if kernel is not None else self._default_kernel()
        self.state = self._init_grid().to(device)

    def _init_grid(self):
        return torch.randint(0, 2, (self.batch_size, 1, *self.grid_size),␣
 ↪dtype=torch.float32)

    def _default_kernel(self):
        k = torch.tensor([[1., 1., 1.],
                          [1., 0., 1.],
                          [1., 1., 1.]], dtype=torch.float32)
        return k.view(1, 1, 3, 3).to(device)
```

```
    def step(self):
        neighbors = F.conv2d(self.state, self.kernel, padding=1)
        self.state = torch.where(neighbors == self.threshold, 1.0, 0.0)

    def mutate_kernel(self, mutation_rate=0.1):
        with torch.no_grad():
            mask = torch.rand_like(self.kernel) < mutation_rate
            self.kernel[mask] = 1.0 - self.kernel[mask]  # flip kernel bits

    def inject_pattern(self, pattern_tensor, position=(0, 0)):
        x, y = position
        h, w = pattern_tensor.shape[-2:]
        self.state[:, :, x:x+h, y:y+w] = pattern_tensor.to(device)

    def compute_entropy(self):
        p = torch.mean(self.state)
        if p == 0 or p == 1:
            return 0.0
        return -(p * torch.log2(p) + (1 - p) * torch.log2(1 - p)).item()

    def run(self, steps=10, mutate_every=None):
        history = [self.state.clone()]
        metrics = []

        for step in range(steps):
            entropy = self.compute_entropy()
            activity = self.state.mean().item()
            metrics.append({'step': step, 'entropy': entropy, 'activity':␣
↪activity})

            if mutate_every and step % mutate_every == 0 and step != 0:
                self.mutate_kernel(mutation_rate=0.05)

            self.step()
            history.append(self.state.clone())

        return torch.cat(history, dim=0), pd.DataFrame(metrics)
```

[9]:
```
# Cell 3: Helper to generate structured patterns (for input injection)

def create_cross_pattern(size=5):
    pattern = torch.zeros((1, 1, size, size))
    pattern[0, 0, size // 2, :] = 1
    pattern[0, 0, :, size // 2] = 1
    return pattern
```

[10]:
```
# Cell 4: Set up CA, inject pattern, and simulate
ca = CellularAutomaton2D(grid_size=(64, 64), batch_size=1, threshold=3)

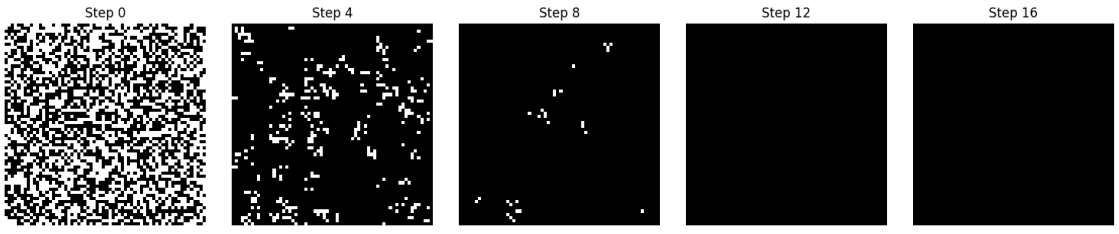
# Inject a cross into the center of the grid
pattern = create_cross_pattern(size=5)
ca.inject_pattern(pattern, position=(29, 29))

# Run with optional mutation
history, metrics = ca.run(steps=20, mutate_every=10)
print(" History shape:", history.shape)
print(metrics.head())
```

```
 History shape: torch.Size([21, 1, 64, 64])
   step   entropy  activity
0     0  0.999938  0.495361
1     1  0.798257  0.241943
2     2  0.678858  0.179443
3     3  0.559123  0.130615
4     4  0.405029  0.080811
```

[11]:
```
# Cell 5: Visualize CA evolution over time
def plot_history(history, interval=5):
    steps = history.shape[0]
    fig, axs = plt.subplots(1, steps // interval, figsize=(15, 3))
    for i, ax in enumerate(axs):
        t = i * interval
        ax.imshow(history[t, 0].cpu(), cmap='gray')
        ax.set_title(f"Step {t}")
        ax.axis('off')
    plt.tight_layout()
    plt.show()

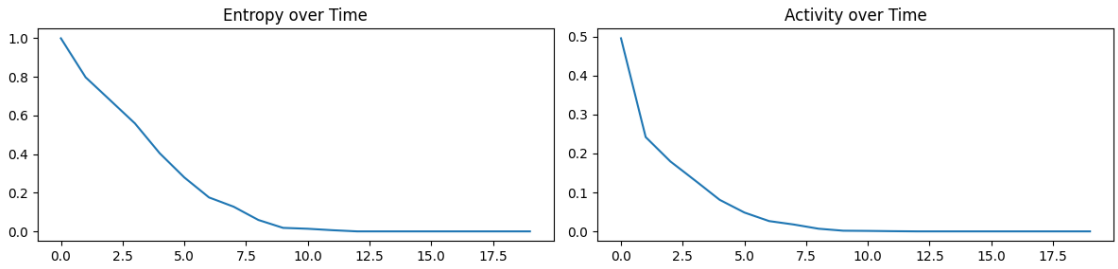
plot_history(history, interval=4)
```



[12]:
```
# Cell 6: Plot entropy and activity over time
def plot_metrics(metrics_df):
    fig, axs = plt.subplots(1, 2, figsize=(12, 3))
    axs[0].plot(metrics_df['step'], metrics_df['entropy'])
```

```
    axs[0].set_title("Entropy over Time")
    axs[1].plot(metrics_df['step'], metrics_df['activity'])
    axs[1].set_title("Activity over Time")
    plt.tight_layout()
    plt.show()

plot_metrics(metrics)
```



1.1.2 Phase 2: Attention Over CA States

[13]:
```
# Cell 1: Imports for attention
import torch.nn as nn
import math

# Optional sinusoidal positional encoding
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=100):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float32).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.
 ↪log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.pe = pe.unsqueeze(0)  # Shape: [1, T, d_model]

    def forward(self, x):
        return x + self.pe[:, :x.size(1)].to(x.device)
```

[14]:
```
# Cell 2: Transformer over CA history with projection head
class CAAttentionModel(nn.Module):
    def __init__(self, input_dim, hidden_dim=128, num_heads=4, num_layers=2,␣
 ↪output_dim=64):
        super().__init__()
        self.input_proj = nn.Linear(input_dim, hidden_dim)
        self.pos_encoder = PositionalEncoding(hidden_dim)
```

```
        encoder_layer = nn.TransformerEncoderLayer(d_model=hidden_dim,
                                                   nhead=num_heads,
                                                   dim_feedforward=hidden_dim*2,
                                                   dropout=0.1,
                                                   batch_first=True)

        self.transformer = nn.TransformerEncoder(encoder_layer,␣
↪num_layers=num_layers)
        self.output_proj = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):  # x: [B, T, H*W]
        x = self.input_proj(x)                    # [B, T, hidden]
        x = self.pos_encoder(x)                   # add positional encoding
        x = self.transformer(x)                   # [B, T, hidden]
        return self.output_proj(x)                # [B, T, output_dim]
```

[15]:
```
# Cell 3: Convert CA [T, 1, H, W] → [1, T, H*W]
def prepare_history_tensor(history):
    T, _, H, W = history.shape
    x = history.view(T, -1).unsqueeze(0)  # Shape: [1, T, H*W]
    return x.to(device)
```

[16]:
```
# Cell 4: Instantiate and run the model
input_tensor = prepare_history_tensor(history)  # [1, T, H*W]
model = CAAttentionModel(input_dim=64*64, output_dim=32).to(device)
output = model(input_tensor)  # [1, T, 32]

print("  Output shape:", output.shape)
```

```
 Output shape: torch.Size([1, 21, 32])
```

[17]:
```
# Cell 5: Define loss and a single training step
def train_step(model, optimizer, x, target, loss_fn):
    model.train()
    optimizer.zero_grad()
    output = model(x)           # [B, T, output_dim]
    pred = output[:, -1, :]     # predict final step from prior context
    loss = loss_fn(pred, target)
    loss.backward()
    optimizer.step()
    return loss.item()
```

[18]:
```
# Cell 6: Training config and loop
from torch.utils.data import DataLoader, TensorDataset

# Hyperparameters
```

```
epochs = 100
learning_rate = 1e-3
input_dim = 64 * 64
output_dim = input_dim
hidden_dim = 128

# Prepare model
model = CAAttentionModel(input_dim=input_dim, hidden_dim=hidden_dim,␣
 ↪output_dim=output_dim).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
loss_fn = nn.BCEWithLogitsLoss()  # binary CA output, logits from model

# Prepare data
x = prepare_history_tensor(history[:-1])          # [1, T, H*W] up to T-1
target = history[-1].view(1, -1).to(device)       # [1, H*W] → step T

# Optional: repeat to simulate a dataset
x_train = x.repeat(16, 1, 1)      # [B, T, H*W]
target_train = target.repeat(16, 1)  # [B, H*W]
dataset = TensorDataset(x_train, target_train)
loader = DataLoader(dataset, batch_size=4, shuffle=True)

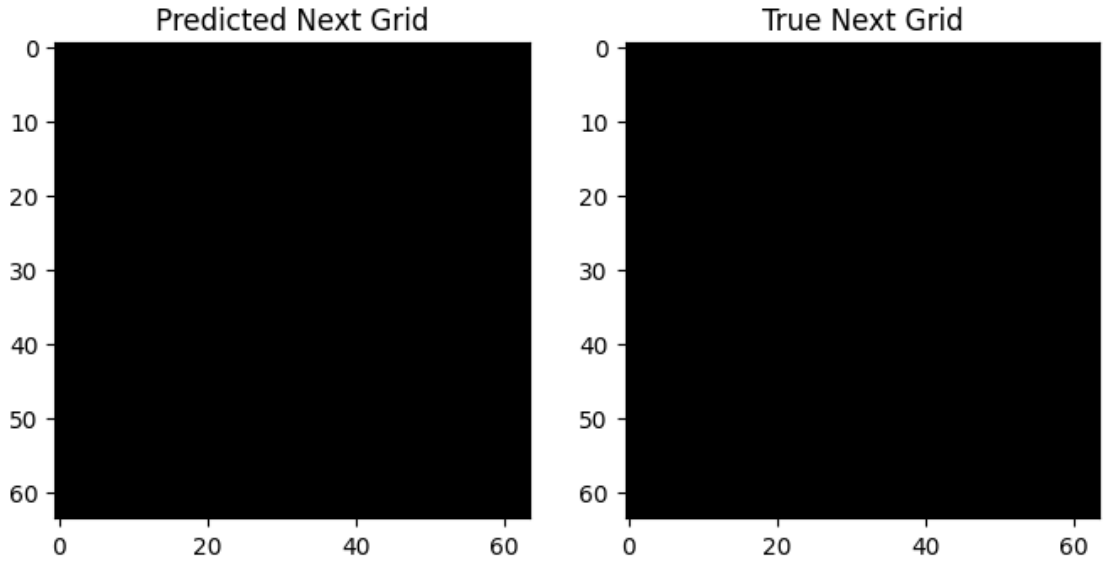
# Training loop
for epoch in range(epochs):
    total_loss = 0.0
    for xb, yb in loader:
        loss = train_step(model, optimizer, xb, yb, loss_fn)
        total_loss += loss
    if epoch % 10 == 0:
        print(f"Epoch {epoch:03d} | Loss: {total_loss / len(loader):.4f}")
```

```
Epoch 000 | Loss: 0.6514
Epoch 010 | Loss: 0.0279
Epoch 020 | Loss: 0.0091
Epoch 030 | Loss: 0.0050
Epoch 040 | Loss: 0.0032
Epoch 050 | Loss: 0.0022
Epoch 060 | Loss: 0.0016
Epoch 070 | Loss: 0.0012
Epoch 080 | Loss: 0.0009
Epoch 090 | Loss: 0.0007
```

[19]:
```
# Cell 7: Evaluate prediction vs. ground truth
model.eval()
with torch.no_grad():
    pred_logits = model(x)[0, -1, :]       # [H*W]
    pred_probs = torch.sigmoid(pred_logits).view(64, 64)
```

```
    pred_binary = (pred_probs > 0.5).float()

# Plot prediction vs ground truth
fig, axs = plt.subplots(1, 2, figsize=(8, 4))
axs[0].imshow(pred_binary.cpu(), cmap='gray')
axs[0].set_title("Predicted Next Grid")
axs[1].imshow(history[-1, 0].cpu(), cmap='gray')
axs[1].set_title("True Next Grid")
plt.show()
```



Generalization with Dataset Gen

[24]:
```
def generate_ca_dataset(
    num_samples=100,
    steps=20,
    grid_size=(64, 64),
    pattern_fn=None,
    kernel=None,
    threshold=3,
    min_alive_threshold=10,
    max_attempts=1000
):
    X, Y = [], []
    attempts = 0

    while len(X) < num_samples and attempts < max_attempts:
        ca = CellularAutomaton2D(grid_size=grid_size, kernel=kernel,␣
↪threshold=threshold)
```

```
        # Inject pattern if provided
        if pattern_fn:
            pattern = pattern_fn()
            x = np.random.randint(0, grid_size[0] - pattern.shape[-2])
            y = np.random.randint(0, grid_size[1] - pattern.shape[-1])
            ca.inject_pattern(pattern, position=(x, y))

        history, _ = ca.run(steps=steps)
        final_alive = history[-1].sum().item()

        if final_alive > min_alive_threshold:
            X.append(history[:-1].view(steps, -1))
            Y.append(history[-1].view(-1))

        attempts += 1

    if len(X) == 0:
        raise RuntimeError("  Could not generate any valid CA samples. Try␣
 ↪lowering threshold or pattern size.")

    print(f"  {len(X)} valid samples collected in {attempts} attempts")
    return torch.stack(X).to(device), torch.stack(Y).to(device)
```

[25]:
```
# Cell 2: Sample pattern functions for injecting activity

def create_cross_pattern(size=5):
    pattern = torch.zeros((1, 1, size, size))
    pattern[0, 0, size // 2, :] = 1
    pattern[0, 0, :, size // 2] = 1
    return pattern

def create_checkerboard(size=8):
    pattern = torch.from_numpy(((np.indices((size, size)).sum(axis=0)) % 2).
 ↪astype(np.float32))
    return pattern.view(1, 1, size, size)
```

[26]:
```
# Cell 3: Generate dataset with random pattern injections
pattern_fn = create_cross_pattern  # try also create_checkerboard
custom_kernel = torch.tensor([
    [0., 1., 0.],
    [1., 1., 1.],
    [0., 1., 0.]
]).view(1, 1, 3, 3).to(device)

X_train, Y_train = generate_ca_dataset(
    num_samples=128,
```

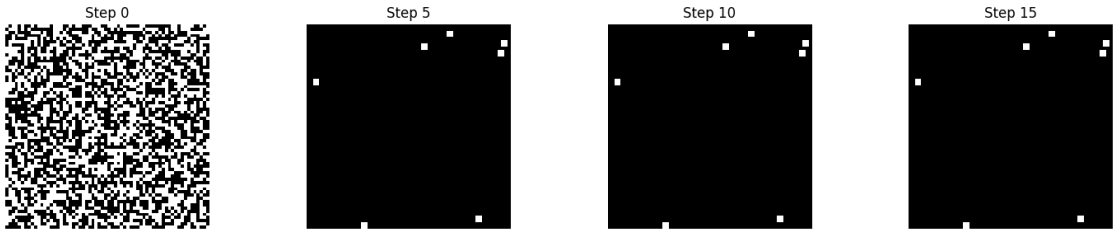
```
    steps=20,
    pattern_fn=create_cross_pattern,
    kernel=custom_kernel,
    threshold=3,                    # more forgiving than 4
    min_alive_threshold=25,       # ensures enough structure
    max_attempts=2000
)


print(" Dataset shape:", X_train.shape, Y_train.shape)  # [B, T, H*W], [B, H*W]

# Plot a few CA histories
def show_example_sequence(x_tensor):
    steps = x_tensor.shape[0]
    fig, axs = plt.subplots(1, steps // 5, figsize=(15, 3))
    for i, ax in enumerate(axs):
        step = i * 5
        grid = x_tensor[step].view(64, 64).cpu()
        ax.imshow(grid, cmap='gray')
        ax.set_title(f"Step {step}")
        ax.axis('off')
    plt.tight_layout()
    plt.show()

show_example_sequence(X_train[0])
```

```
 18 valid samples collected in 2000 attempts
 Dataset shape: torch.Size([18, 20, 4096]) torch.Size([18, 4096])
```



[27]:
```
# Cell 4: Train Transformer model to generalize across varied runs

model = CAAttentionModel(input_dim=64*64, hidden_dim=128, output_dim=64*64).
 ↪to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.BCEWithLogitsLoss()

dataset = torch.utils.data.TensorDataset(X_train, Y_train)
```

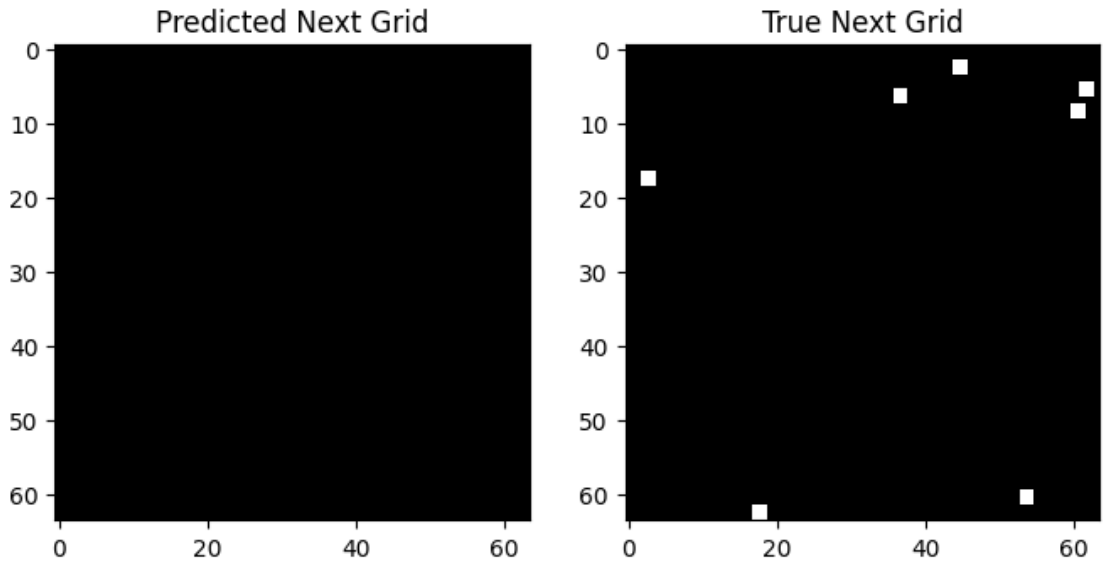
```
loader = torch.utils.data.DataLoader(dataset, batch_size=8, shuffle=True)

for epoch in range(30):
    total_loss = 0.0
    for xb, yb in loader:
        model.train()
        optimizer.zero_grad()
        pred = model(xb)[:, -1, :]  # predict last step
        loss = loss_fn(pred, yb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    if epoch % 5 == 0:
        print(f"Epoch {epoch:02d} | Loss: {total_loss / len(loader):.4f}")
```

```
Epoch 00 | Loss: 0.6695
Epoch 05 | Loss: 0.1923
Epoch 10 | Loss: 0.0730
Epoch 15 | Loss: 0.0482
Epoch 20 | Loss: 0.0402
Epoch 25 | Loss: 0.0374
```

[28]:
```
# Cell 5: Visualize prediction vs true next state
model.eval()
with torch.no_grad():
    test_input = X_train[0:1]                  # [1, T, H*W]
    test_true = Y_train[0].view(64, 64)        # [H, W]
    test_pred = torch.sigmoid(model(test_input)[0, -1]).view(64, 64)
    test_pred_bin = (test_pred > 0.5).float()

# Plot side by side
fig, axs = plt.subplots(1, 2, figsize=(8, 4))
axs[0].imshow(test_pred_bin.cpu(), cmap='gray')
axs[0].set_title("Predicted Next Grid")
axs[1].imshow(test_true.cpu(), cmap='gray')
axs[1].set_title("True Next Grid")
plt.show()
```



1.1.3 Phase 3: Symbolic Logic Integration

Phase 3 Prep A: Attention map visualizations

[37]:
```
class TransformerEncoderLayerWithAttn(nn.TransformerEncoderLayer):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.attn_weights = None

    def forward(self, src, src_mask=None, src_key_padding_mask=None,␣
 ↪is_causal=False):
        src2, attn_weights = self.self_attn(
            src, src, src, attn_mask=src_mask,
            key_padding_mask=src_key_padding_mask,
            need_weights=True,
            average_attn_weights=False
        )
        self.attn_weights = attn_weights.detach().cpu()
        src = src + self.dropout1(src2)
        src = self.norm1(src)
        src2 = self.linear2(self.dropout(self.activation(self.linear1(src))))
        src = src + self.dropout2(src2)
        src = self.norm2(src)
        return src
```

[42]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
def plot_attention(attn_maps, layer=0, head=0, step=None):
    """
    Visualize attention map for a specific layer, head, and optionally query␣
 ↪step.
    If step is None, shows the full matrix.
    """
    attn = attn_maps[layer][0, head].numpy()  # shape [T, T]

    if step is not None:
        attn_row = attn[step]  # attention from this step
        plt.figure(figsize=(10, 1))
        sns.heatmap(attn_row[None, :], cmap='viridis', cbar=True,␣
 ↪xticklabels=False, yticklabels=False)
        plt.title(f"Layer {layer} | Head {head} | Attention from Step {step}")
        plt.xlabel("Key (Time Step)")
        plt.ylabel("Query")
    else:
        plt.figure(figsize=(6, 5))
        sns.heatmap(attn, cmap='viridis', xticklabels=False, yticklabels=False)
        plt.title(f"Layer {layer} | Head {head} | Full Attention Map")
        plt.xlabel("Key (Time Step)")
        plt.ylabel("Query (Time Step)")

    plt.tight_layout()
    plt.show()
```

[43]:
```
class CAAttentionModel(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim, n_layers=2,␣
 ↪n_heads=4):
        super().__init__()
        self.embed = nn.Linear(input_dim, hidden_dim)

        self.encoder_layers = nn.ModuleList([
            TransformerEncoderLayerWithAttn(
                d_model=hidden_dim,
                nhead=n_heads,
                dim_feedforward=hidden_dim * 4,
                batch_first=True
            ) for _ in range(n_layers)
        ])
        self.encoder = nn.Sequential(*self.encoder_layers)
        self.proj = nn.Linear(hidden_dim, output_dim)

    def forward(self, x, return_attn=False):
        x = self.embed(x)
        for layer in self.encoder_layers:
            x = layer(x)
```

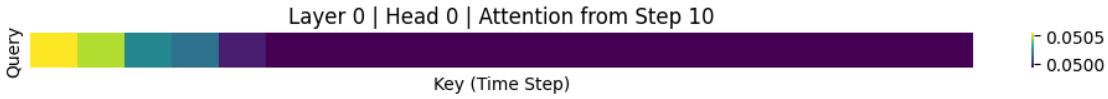
```
        out = self.proj(x)

        if return_attn:
            # collect all attention maps: [layers][batch, heads, T, T]
            attn_maps = [layer.attn_weights for layer in self.encoder_layers]
            return out, attn_maps
        return out
```

[44]:
```
model = CAAttentionModel(input_dim=64*64, hidden_dim=128, output_dim=64*64).
↪to(device)
```

[45]:
```
test_input = X_train[0:1]
model.eval()
with torch.no_grad():
    output, attn_maps = model(test_input, return_attn=True)
```

[46]:
```
plot_attention(attn_maps, layer=0, head=0, step=10)
```



1.1.4 Phase 3N

[47]:
```
import torch

def extract_causal_rules(attn_maps, threshold=0.05):
    """
    From attn_maps, extract a causal influence map: which steps influence which.
    Returns a binary matrix [T, T] where 1 means causal edge from j→i (j␣
↪influences i)
    """
    # attn_maps: [num_layers][batch, num_heads, T, T]
    layer = 0  # we'll start with layer 0 for now
    head_attn = attn_maps[layer][0]  # shape [num_heads, T, T]

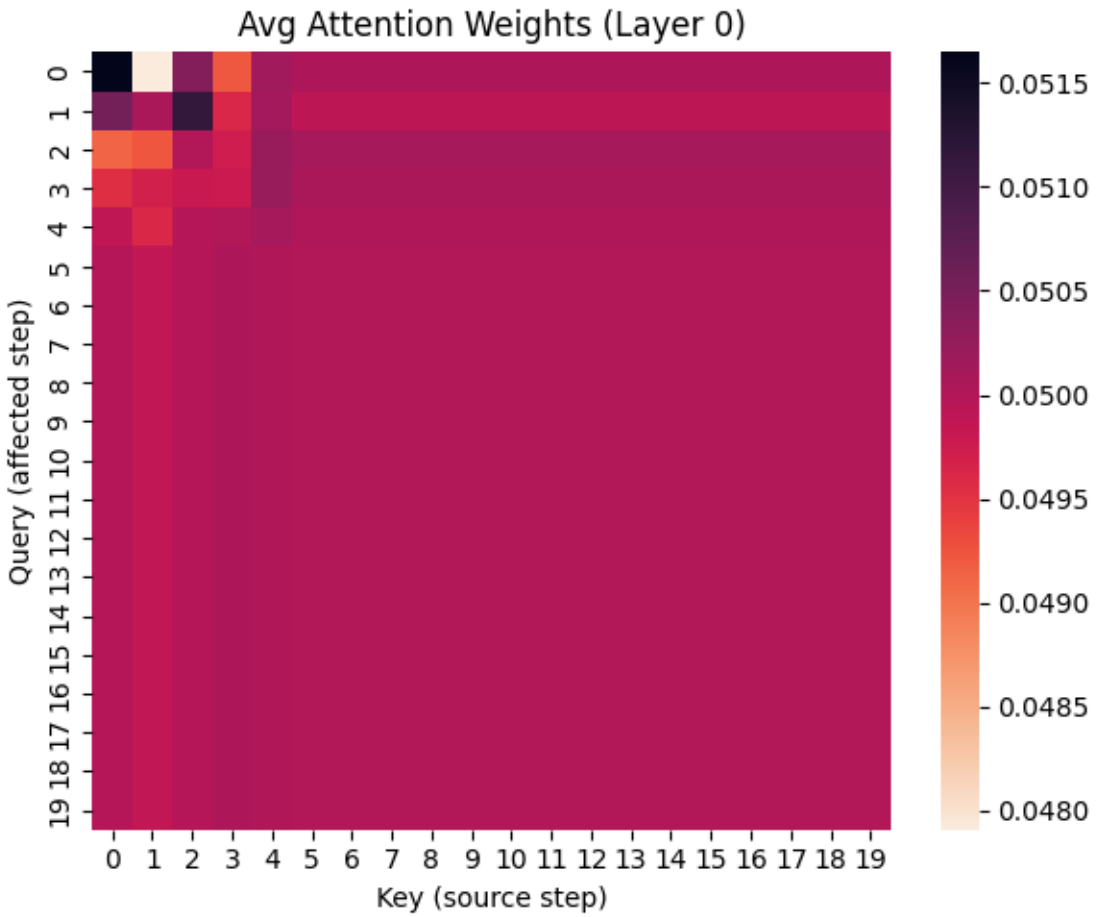
    avg_attn = head_attn.mean(dim=0)  # shape [T, T], average over heads

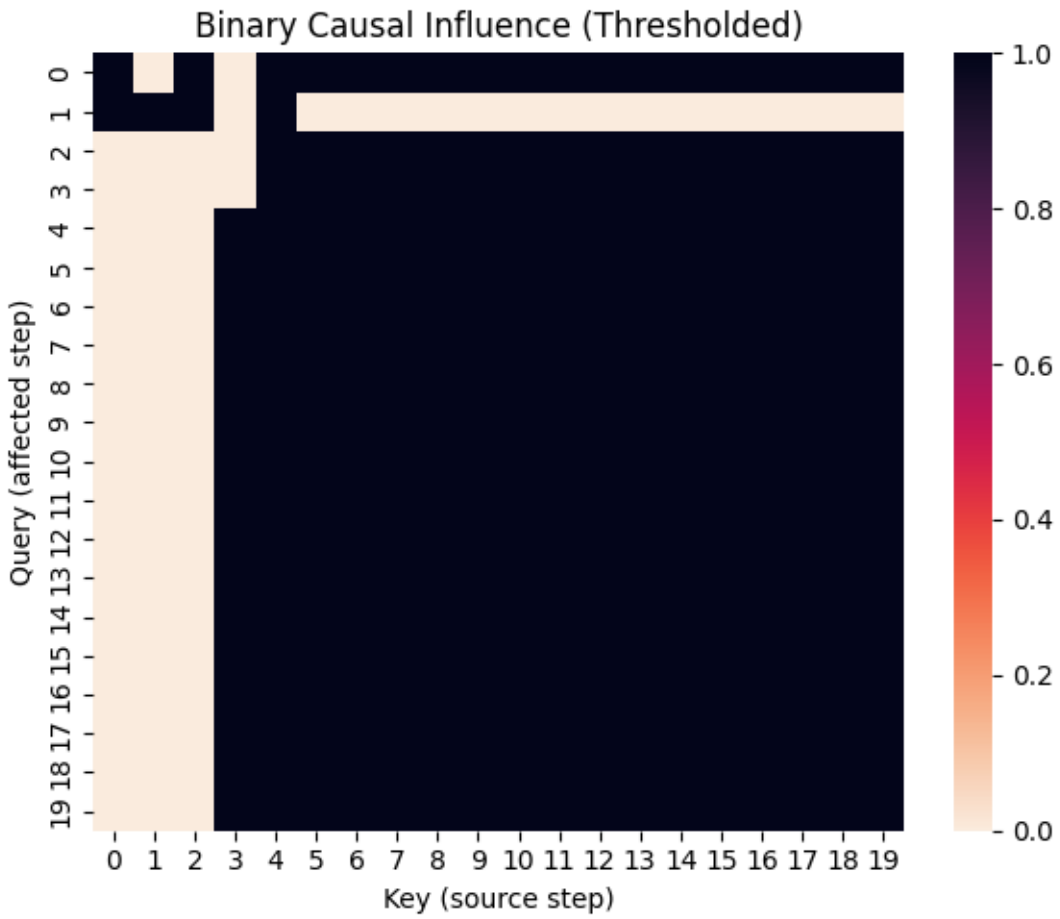
    causal_edges = (avg_attn > threshold).int()  # binary causal matrix
    return avg_attn, causal_edges
```

[48]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
def plot_causal_matrix(causal_matrix, title="Causal Influence (Query ← Key)"):
    """
    Plots a heatmap where row i, column j means step j influences step i
    """
    plt.figure(figsize=(6, 5))
    sns.heatmap(causal_matrix, cmap="rocket_r", cbar=True)
    plt.title(title)
    plt.xlabel("Key (source step)")
    plt.ylabel("Query (affected step)")
    plt.tight_layout()
    plt.show()
```

[49]:
```
avg_attn, causal_matrix = extract_causal_rules(attn_maps, threshold=0.05)
plot_causal_matrix(avg_attn, "Avg Attention Weights (Layer 0)")
plot_causal_matrix(causal_matrix, "Binary Causal Influence (Thresholded)")
```





[50]:
```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
```

[58]:
```
# Average across heads if needed
avg_attn_weights = attn_maps[0].mean(dim=0)  # shape: [T, T]

# Threshold to binary influence matrix
threshold = 0.051  # or pick dynamically based on mean/std
binary_attn = (avg_attn_weights > threshold).int()
```

[59]:
```
def build_temporal_graph(binary_attn_matrix):
    """
    Constructs a directed temporal graph from a binary attention matrix.
    Args:
        binary_attn_matrix: [T, T] binary matrix (NumPy or torch.Tensor)
    Returns:
        G: NetworkX DiGraph
```

```
    """
    if hasattr(binary_attn_matrix, "cpu"):
        binary_attn_matrix = binary_attn_matrix.cpu().numpy()

    T = binary_attn_matrix.shape[0]
    G = nx.DiGraph()
    G.add_nodes_from(range(T))

    for i in range(T):
        for j in range(T):
            if binary_attn_matrix[i, j] == 1 or binary_attn_matrix[i, j] > 0:
                G.add_edge(j, i)  # j → i = key → query

    return G
```

[65]:
```
import matplotlib.pyplot as plt
import networkx as nx

def plot_linear_temporal_graph(G, title="Linear Temporal Causal Graph"):
    """
    Plots a DAG with nodes arranged left-to-right by time.
    """
    pos = {i: (i, 0) for i in G.nodes}
    plt.figure(figsize=(10, 2))
    nx.draw(G, pos,
            with_labels=True,
            arrows=True,
            node_color='lightblue',
            edge_color='gray',
            node_size=400,
            font_size=10)
    plt.title(title)
    plt.show()
```

[66]:
```
def plot_temporal_graph(G, title="Temporal Causal Graph"):
    pos = nx.spring_layout(G, seed=42)
    plt.figure(figsize=(6, 4))
    nx.draw(G, pos, with_labels=True, arrows=True, node_color='lightblue',␣
 ↪edge_color='gray')
    plt.title(title)
    plt.show()
```

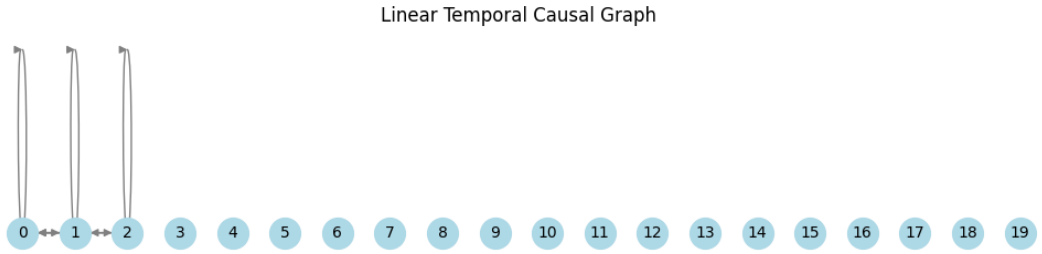
[67]:
```
print("binary_attn shape:", binary_attn.shape)
print("Example entry shape:", binary_attn[0, 0].shape)
```

```
binary_attn shape: torch.Size([4, 20, 20])
Example entry shape: torch.Size([20])
```

[68]:
```
# Collapse across heads → [T, T]
binary_attn_matrix = binary_attn.float().mean(dim=0)  # or .max(dim=0).values

# Threshold to binarize
threshold = 0.05
binary_attn_matrix = (binary_attn_matrix > threshold).int()
```

[69]:
```
G = build_temporal_graph(binary_attn_matrix)
plot_linear_temporal_graph(G)
```



1.1.5 Phase 4: Recursive Self-Modification

1.1.6 Phase 5: Evaluation and Benchmarks